6-WEEK SHARING SERIES

GitHub for UI Designers

用 AI 時代的方式，帶設計師認識 Git

Learn Git through the lens of AI-powered design workflows

Week 1 TODAY	**為什麼設計師要學 Git？** Why Git for Designers?	Git vs Figma 協作模式差異、核心觀念轉換 Collaboration model comparison & mindset shift
Week 2	**Git 的基本架構** Git Architecture	Git vs GitHub、三個空間、本機 vs 遠端 Git vs GitHub, three spaces, local vs remote
Week 3	**Git 常見名詞解惑** Git Vocabulary	Commit vs Push、Branch vs Fork、Figma 類比總整理 Key terms explained with Figma analogies
Week 4	**用 AI 操作 Git** Using AI for Git	Claude Code 操作示範、安全機制、避免 push 錯路徑 Natural language → Git commands, safety mechanisms
Week 5	**Repo 怎麼分層管理？** Repo Taxonomy	CLAUDE.md、四層 repo 架構、Design System 歸屬 4-tier repo structure & where Design System belongs
Week 6	**多專案工作流** Multi-Repo Workflow	一人多 repo 架構、每日流程、Global CLAUDE.md Daily workflow across multiple repos

Week 1 / 6

為什麼設計師要學 Git？

Why Git for Designers?

GitHub for UI Designers

The Problem

今年起，我們全面採用 AI Design 工作流

This year, we fully adopted an AI-powered design workflow.

但是...

But...

1

AI 工具（如 Claude Code）直接產出程式碼
需要用 Git 管理版本

AI tools (like Claude Code) generate real code that needs version control

2

設計師需要 push 設計稿到 repo
但不熟悉 Git 的運作邏輯

Designers need to push designs to repos but aren't familiar with how Git works

3

多人同時用 AI 產出 → 合併衝突
不知道怎麼處理

Multiple people generating with AI → merge conflicts with no idea how to resolve them

TODAY'S GOAL

~~今天不會教你打指令~~

~~Today we won't teach you any commands~~

我們要聊的是：

Git 的「思考方式」
跟 Figma 有什麼不同？

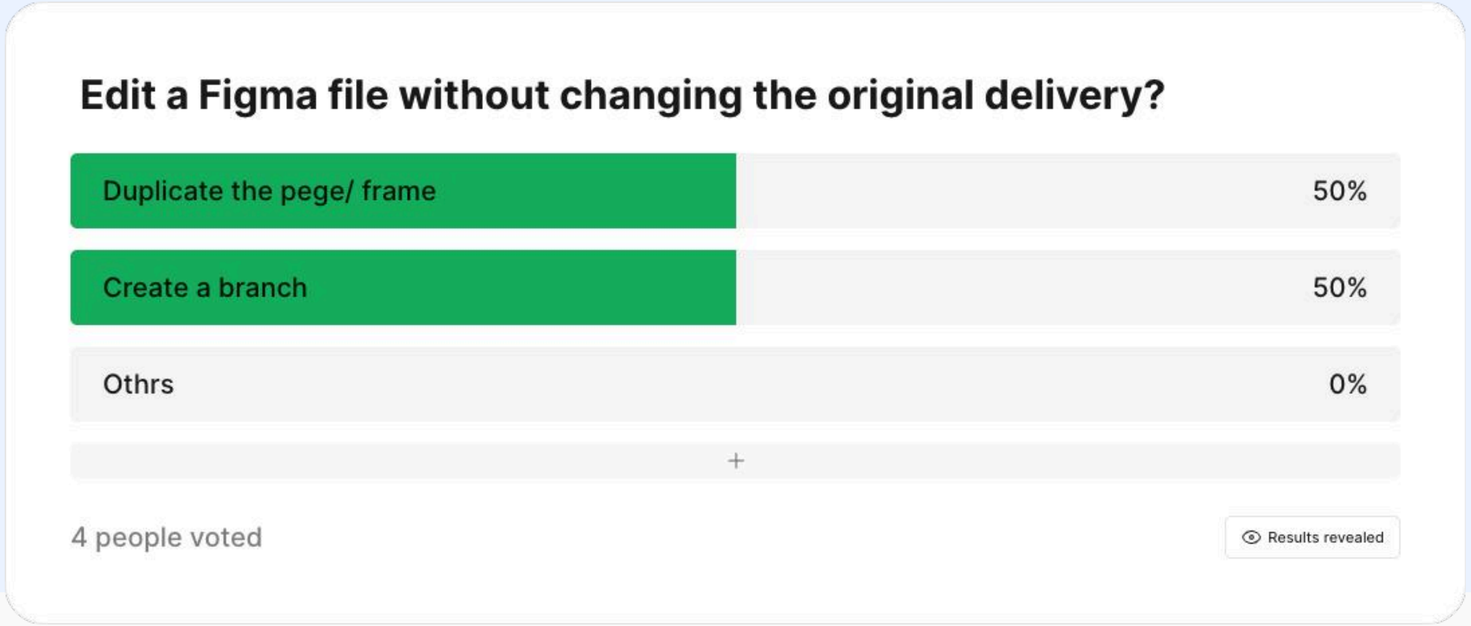
How does Git's mental model differ from Figma's?

?

QUICK QUESTION

當你不想影響原始檔案，
你在 Figma 會怎麼做？

When you want to experiment without affecting original files,
what do you do in Figma?



COMPARISON

Git vs Figma — 協作模式的差異

Collaboration Model Comparison (1/2)

	Figma	Git
同步方式 Sync	**瀏覽器 ↔ 雲端（即時自動）** Browser/ client ↔ Cloud (real-time)	**本機 ↔ GitHub（手動 pull / push）** Local ↔ GitHub (manual)
協作模型 Collaboration	**多人同時編輯同一份** Everyone edits the same file	**每人有自己的副本，改完再合併** Each person has a copy, merge later
可見性 Visibility	**改動即可見** Changes visible instantly	**你決定什麼時候讓別人看到** You decide when to share
存檔 Saving	**自動儲存** Auto-save	**手動決定存什麼（commit）** You decide what to save (commit)

COMPARISON

Git vs Figma — 協作模式的差異

Collaboration Model Comparison (2/2)

	Figma	Git
版本歷史 Version History	**系統自動記錄** Auto-recorded by system	**commit message 就是版本歷史** Commit messages ARE the history
錯誤回復 Recovery	**Undo / Version History** Undo / Version History	**需主動建 Branch 保護** Requires proactive branching
衝突處理 Conflict	**系統自動合併，衝突不可見** Auto-merged, conflicts invisible	**Merge conflict 需手動解決** Manual conflict resolution
離線工作 Offline	**需要網路** Requires internet	**可離線 commit，之後再 push** Commit offline, push later
平行工作流 Parallel Work	**Figma Branch（付費功能）** Figma Branch (paid feature)	**Branch 是免費的核心功能** Branch is free & fundamental

SCENARIO 01

不想影響別人，自己先改改看

Experiment without affecting others

Figma

Duplicate page/ frame/
Create a branch (paid)

Copy the entire page/ frame to try things out

Git

開一條 Branch

Create a branch — your own parallel workspace

SCENARIO 02

改好了，交付給團隊

Ready to hand off changes to the team

Figma

把 draft 移回 main page
＋ 留言通知

Move draft back to main page + leave a comment

Git

commit → push
→ 開 Pull Request

Commit → Push → Open a Pull Request for review

SCENARIO 03

同事改壞了，想回到之前的版本

Something broke, need to go back

Figma

Version History 找回

Browse version history to restore

Git

git log 找到 commit
→ git revert

Find the commit in log → revert it

KEY INSIGHT

兩種不同的設計哲學

Two Different Philosophies

Figma

即時
Instant

自動
Automatic

所見即所得
WYSIWYG

vs

Git

刻意
Intentional

手動
Manual

你來決定
You Decide

KEY INSIGHT

在 Git 的世界，你掌控一切

In Git, you're in control

1

什麼時候存

When to save

commit

2

存什麼

What to save

staging

3

何時讓別人看

When to share

push

4

怎麼合併

How to combine

merge / PR

?

QUICK QUESTION

聽起來 Git 比 Figma 麻煩很多，
為什麼不能像 Figma 一樣即時同步就好？

Git sounds way more complicated.
Why can't it just sync in real-time like Figma?

THE ANSWER

因為管理的東西不一樣

Because they manage different things

Figma **An App**

管的是「視覺畫面」

Manages visual layouts

↓

改了馬上看到結果
即時同步很安全

Changes are immediately visible.
Real-time sync is safe.

Git **A version tracker**

管的是「程式碼」

Manages source code

↓

需要編譯才能執行
改到一半被 pull = 壞掉

Code must compile to run.
Half-finished code pulled by others = broken.

TAKEAWAY

~~Git 不是更難的 Figma~~

~~Git is not a harder version of Figma~~

它是不同的協作哲學

It's a different collaboration philosophy

Figma：讓工具幫你管理一切
Figma: Let the tool manage everything

Git：讓你有意識地管理每一步
Git: Manage every step intentionally

NEXT WEEK

Week 2 / 6

Git 的基本架構

Git Architecture

Git vs GitHub — 到底差在哪？

Git vs GitHub — What's the difference?

三個空間 — 程式碼的旅程

Three spaces — The journey of your code

本機 vs 遠端

Local vs Remote

See you next week!